

INFO8006 Introduction to Artificial Intelligence

Exercise session 1: Solving problems by searching

RECAP

Search problem

A *search problem* is defined by a representation of the *states*, an *initial state* s_0 , the *actions* $\text{actions}(s)$ available in a state s , a *transition model* $s' = \text{result}(s, a)$ and a *goal test*. Together, these define the *state space*, the set of states reachable from s_0 . A *solution* is an action sequence $a_1, \dots, a_T$ leading from s_0 to a goal state s_T ; an *optimal solution* has the lowest *path cost* $\sum_{t=1}^T c(s_{t-1}, a_t, s_t)$, a sum of positive *step costs* c .

Search algorithm

Tree search expands the nodes of the *fringe* until a goal state is reached. *Graph search* also keeps the set of expanded states, the *closed* set, which avoids repeated states and cycles.

Search strategies

A *strategy* is the order in which the fringe is expanded, where $g(n)$ is the path cost from s_0 to n , $h(n)$ a heuristic and $f(n) = g(n) + h(n)$. Below, b is the maximum branching factor, d the depth of the least-cost solution, m the maximum path length (possibly infinite), C^* the optimal path cost and $\epsilon > 0$ a lower bound on the step costs.

Strategy	Fringe	Complete	Optimal	Time	Space
Depth-first (DFS)	LIFO stack	if no cycles	no	$O(b^m)$	$O(bm)$
Breadth-first (BFS)	FIFO queue	yes	if c uniform	$O(b^d)$	$O(b^d)$
Iterative deepening	DFS, depth 1, 2, ...	yes	if c uniform	$O(b^d)$	$O(bd)$
Uniform-cost (UCS)	priority $g(n)$	if $c \geq \epsilon$	yes	$O(b^{C^*/\epsilon})$	$O(b^{C^*/\epsilon})$
Greedy	priority $h(n)$	if no cycles	no	$O(b^m)$	$O(b^m)$
A*	priority $f(n)$	yes	if h admissible	$O(b^m)$	$O(b^m)$

Heuristics

A *heuristic* $h(n) \geq 0$ estimates the cost of the cheapest path from n to a goal state, with $h(n) = 0$ at a goal state, often as the exact cost of a *relaxed problem*. Let $h^*(n)$ be the true cost from n to a nearest goal.

- *Admissible*: $h(n) \leq h^*(n)$ for all n , i.e. h never overestimates. Tree-search A* is then optimal.
- *Consistent*: $h(n) \leq c(n, a, n') + h(n')$ for every n and every successor n' generated by an action a . Then h is admissible, f is non-decreasing along any path and graph-search A* is optimal.
- *Dominance*: if h_1 and h_2 are admissible and $h_2(n) \geq h_1(n)$ for all n , then h_2 dominates h_1 and is better for search. $\max(h_a, h_b)$ is admissible and dominates both.

In session: exercises 1, 3 (a)–(c), 5.

Exercise 1 Path finding

Tram'Ardent maintains the green areas along the Liège tram line with a new robot. This robot's goal is to convey the grass cuttings left by the gardeners to a container as fast as possible. Assuming that

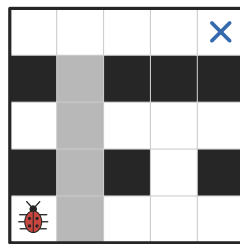
- the fuel cost used by the robot is negligible
- the robot can only navigate on guides that are placed in a rectangular grid configuration and can locate itself on the grid
- there is only one pile and one container by zone (*i.e.* by grid) for which positions are known by the robot
- the speed of the robot is constant and the grass loading/unloading is instantaneous
- the robot can only carry a limited weight of grass at once
- the robot can know the initial weight of grass on the pile.

For a given grid

- (a) Formulate a corresponding search problem.
- (b) Knowing that the terrain can be rugged at specific places, causing slow down of the robot, propose an admissible heuristic which estimates the remaining cost at any node.
- (c) The robot can now also move diagonally, a diagonal step costing $\sqrt{2}$. Is your heuristic still admissible? If not, what should you change? What if the robot is twice as fast diagonally, so that a diagonal step costs $\sqrt{2}/2$?

Exercise 2 The hive (CS188, Spring 2019)

The hive of insects needs your help. You control an insect in a rectangular maze-like environment of size $M \times N$, as shown in the figure below. At each time-step, the insect can move into a free adjacent cell or stay in its current location. All actions have a unit cost.



In this particular case, the insect must pass through a series of partially flooded tunnels, as illustrated by the gray cells on the map. The insect can hold its breath for A time-steps in a row. Moving into a flooded cell requires your insect to consume 1 unit of air, while moving into a free cell refills its air supply.

- (a) Give a minimal state space for this problem (do not include extra information). You should answer for a general instance of the problem, not the specific map above.
- (b) Give the size of your state space.

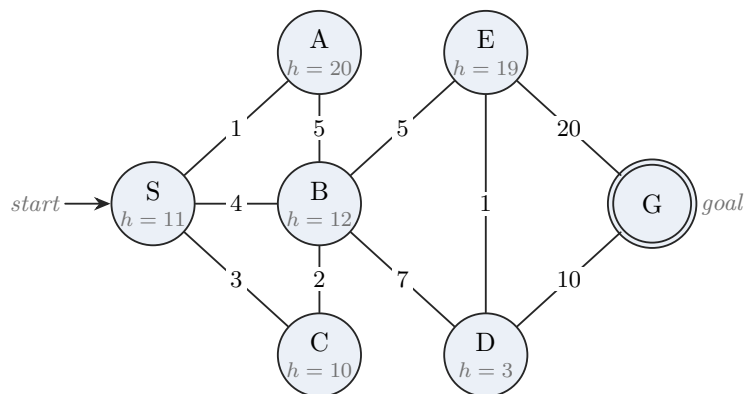
Exercise 3 Graph search

For each of the following search algorithms, apply the GRAPH-SEARCH algorithm (lecture 2, slide 62) to the undirected graph below and give the order in which the states are expanded, as well as the final path returned by the algorithm. Ties are broken by alphabetical order on the states and, between two nodes of the same state, in favour of the one inserted first in the fringe. The goal test is applied when a node leaves the fringe, so the goal state is returned without ever being expanded.

```

function GRAPH-SEARCH(problem, fringe) returns a solution, or failure
  closed  $\leftarrow$  an empty set
  fringe  $\leftarrow$  INSERT(MAKE-NODE(INITIAL-STATE[problem]), fringe)
  loop do
    if fringe is empty then return failure
    node  $\leftarrow$  REMOVE-FRONT(fringe)
    if GOAL-TEST(problem, STATE[node]) then return node
    if STATE[node] is not in closed then
      add STATE[node] to closed
      fringe  $\leftarrow$  INSERTALL(EXPAND(node, problem), fringe)
  end

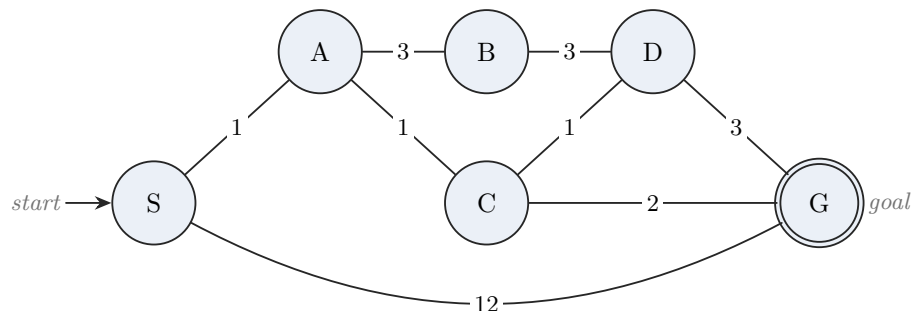
```



- Depth-first search (DFS)
- Breadth-first search (BFS)
- Uniform-cost search (UCS)
- Greedy search
- A*
 - Is the heuristic admissible? If not, make it admissible.
 - Is the heuristic consistent? If not, can we apply GRAPH-SEARCH?
 - Apply A* with the admissible heuristic, using the variant of GRAPH-SEARCH that re-expands a closed state reached with a lower priority.

Exercise 4 Search algorithms

Apply the GRAPH-SEARCH algorithm to the graph below, with the conventions of exercise 3.



- (a) Apply iterative deepening, with a depth limit growing from 1. In which order are the states expanded, which path is returned, and is it optimal?
- (b) Consider the following heuristics:

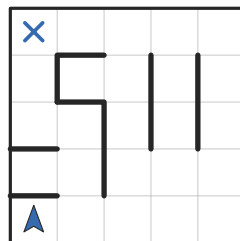
State	h_1	h_2
S	5	4
A	3	2
B	6	6
C	2	1
D	3	3
G	0	0

which one is not admissible? Why?

- (c) Is the admissible heuristic consistent? Does it matter here?
- (d) Apply A* with the admissible heuristic.
- (e) Which of these strategies are guaranteed to return the optimal path on this graph?

Exercise 5 Maze car (CS188, Spring 2014)

Consider a car agent which has to exit a maze. At all time-steps, the agent points at direction $d \in \{N, S, E, W\}$. With a single action, the agent can either move forward at an adjustable velocity $v \in [0, V]$ or turn. The turning actions are **left** and **right**, which change the agent's direction by 90 degrees. Turning is only permitted when the velocity is zero (and leaves it at zero). The moving actions are **faster** and **slower**. The action **faster** increments the velocity by 1 and **slower** decrements the velocity by 1; in both cases the agent then moves a number of squares equal to its *new* velocity. Any action that would result in a collision with a wall is illegal. Any action that would reduce v below 0 or above a maximum speed V is also illegal. The agent's goal is to find a plan which parks it (stationary) on the exit square using as few actions (time steps) as possible.



As an example, in the maze above, if the agent is initially stationary, it might first turn to the east using **right**, then move one square east using **faster**, then two more squares east using **faster** again. However, the agent will have to slow to take the turn.

- (a) For a grid of size $M \times N$, what is the size of the state space? Assume that all configurations are reachable from the starting state.
- (b) What is the maximum branching factor of this problem? Assume that illegal actions are simply not returned by the successor function.
- (c) Is the Manhattan distance from the agent's location to the exit's location admissible?
- (d) If we used an inadmissible heuristic in A* tree search, could it change the completeness of the search? And the optimality?
- (e) State and motivate a non-trivial admissible heuristic for this problem.
- (f) Give a general advantage that an inadmissible heuristic might have over an admissible one.

Exercise 6 Imperfect heuristic (CS188, Spring 2019)

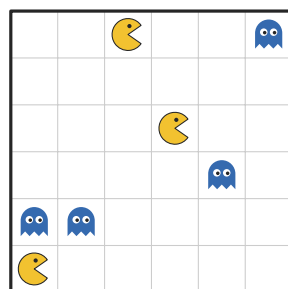
Consider a graph search problem where all edges have a unit cost and the optimal solution has cost C^* . Let $h(n)$ be a heuristic which is $\max\{h^*(n) - k, 0\}$, where $h^*(n)$ is the true forward cost of n and $k \leq C^*$ is a non-negative constant.

- (a) Is h admissible?
- (b) Which of the following is the most reasonable description of how much more work will be done, *i.e.* how many more nodes will be expanded, with heuristic h compared to h^* , as a function of k ?
- ☐ Constant in k
 - ☐ Linear in k
 - ☐ Exponential in k
 - ☐ Unbounded

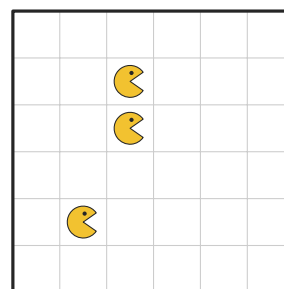
Exercise 7 Pacmen (Exam, January 2023)

Pacman and his friends have decided to combine forces and go on the offensive, and are now chasing ghosts! In a grid of size $M \times N$, Pacman and $P - 1$ of his friends are moving around to collectively eliminate all of the ghosts in the grid by stepping on the same square as each of them. Moving into the same square as a ghost will eliminate it from the grid. At every turn, Pacman and his friends may choose one of the four (**north**, **south**, **east**, **west**) actions but may not collide with each other. In other words, any action that would result in two or more Pacmen occupying the same square will result in no movement for either of the Pacmen. Additionally, Pacman and his friends are *indistinguishable* from each other. There are a total of G ghosts, which are also indistinguishable from each other and cannot move.

Treating this as a search problem, we consider each configuration of the grid to be a state, and the goal state to be the configuration where all ghosts have been eliminated from the board. The cost of each turn (all Pacmen move once) is 1. Below is an example starting state, as well as an example of goal state.



Initial state example



Final state example

Suppose that Pacman has no friends ($P = 1$).

- (a) What is the size of the minimal state space representation? Explain your answer.
- (b) Explain whether each of the following heuristics is admissible, consistent, neither, or both.
- $h_1(s)$ = the sum of the Manhattan distances from Pacman to every remaining ghost.
 - $h_2(s)$ = the number of ghosts times the maximum Manhattan distance between Pacman and any of the ghosts.
 - $h_3(s)$ = the number of remaining ghosts.

Suppose that Pacman has exactly one less friend than the number of ghosts ($P = G$).

- (c) What is the size of the minimal state space representation? Explain your answer.
- (d) Explain whether each of the following heuristics is admissible, consistent, neither, or both.
- $h_4(s)$ = the largest of the Manhattan distances between each Pacman and its closest ghost.
 - $h_5(s)$ = the smallest of the Manhattan distances between each Pacman and its closest ghost.
 - $h_6(s)$ = the number of remaining ghosts.
 - $h_7(s)$ = the number of remaining ghosts divided by P .

QUIZ

1. An agent is rational if ...
 - ☐ its actions always lead to the best possible outcome.
 - ☐ it knows the state s_t of the environment at all times.
 - ☐ its policy maximizes $\mathbb{E}[V(s_{1:T}) \mid \pi]$, given its percepts and prior knowledge.
 - ☐ it maximizes the performance measure that its designer had in mind.
2. As a task environment, poker is ...
 - ☐ discrete, multi-agent and deterministic.
 - ☐ partially observable, stochastic and multi-agent.
 - ☐ partially observable, episodic and single-agent.
 - ☐ None of the above.
3. In a stochastic environment, a simple reflex agent ($a_t = \pi(e_t)$) given the same percept twice will ...
 - ☐ take the same action every time.
 - ☐ take a different action every time.
 - ☐ take a different action most of the time.
 - ☐ take the same action most of the time.
4. A goal-based agent ...
 - ☐ selects its action from the current percept e_t only.
 - ☐ searches its transition model for an action sequence that reaches a state of G .
 - ☐ prefers a fast and safe route over a slow and risky one.
 - ☐ maximizes the expected performance $\mathbb{E}[V(s_{1:T}) \mid e_{1:t}, a_t]$ of its actions.
5. The A^* algorithm, in its graph version (GRAPH-SEARCH, without re-expansion) is ...
 - ☐ always optimal.
 - ☐ always optimal if the heuristic is admissible.
 - ☐ always optimal if the heuristic is consistent.
 - ☐ never optimal when there are cycles.
6. Which of the following statements about heuristics is true?
 - ☐ $h(n) = 0 \ \forall n$ can speed up A^* compared to UCS.
 - ☐ If a heuristic is admissible, it is guaranteed to be consistent.
 - ☐ The average step cost $h(n) = \frac{1}{|\text{actions}(n)|} \sum_a c(n, a, \text{result}(n, a))$ is admissible.
 - ☐ The true remaining cost dominates all admissible heuristics.

7. Iterative deepening is often preferred to breadth-first search because it ...
- ☐ expands strictly fewer nodes.
 - ☐ finds a solution at the same depth d with $O(bd)$ memory, instead of $O(b^d)$.
 - ☐ is optimal even when the step costs differ.
 - ☐ never expands the same state twice.
8. In GRAPH-SEARCH, keeping a closed set ...
- ☐ makes depth-first search complete on a finite state space.
 - ☐ makes any admissible heuristic consistent.
 - ☐ guarantees that A* is optimal whenever h is admissible.
 - ☐ is only useful when the state space is infinite.